

INFORME MAESTRO TÉCNICO — CEN Plataforma de Educación Financiera

Versión: 1.0 | **Fecha:** 2026-05-10 | **Base:** commit 672025e | **Propietario:** Cristofer Huerta Esquivel | **Preparado por:** Auditoría técnica automatizada (Claude Sonnet 4.6)

Índice

- 1. [Resumen Ejecutivo](#)
- 2. [Arquitectura Técnica](#)
- 3. [Estructura del Código](#)
- 4. [Modelo de Datos](#)
- 5. [Autenticación y Autorización](#)
- 6. [Contenido Pedagógico](#)
- 7. [Flujos de Usuario](#)
- 8. [Integraciones Externas](#)
- 9. [Deploy e Infraestructura](#)
- 10. [Estado de Testing y Monitoreo](#)
- 11. [Capacidades Reales del Producto](#)
- 12. [Deuda Técnica Documentada](#)
- 13. [Seguridad y Privacidad](#)
- 14. [Limitaciones de Escalabilidad](#)
- 15. [Historia de Construcción](#)
- 16. [Documentación Existente](#)
- 17. [Recomendaciones para Profesionalización](#)
- 18. [Preguntas Abiertas](#)
- [Apéndice: Inventario de Actividades por Grado](#)

1. Resumen Ejecutivo

CEN (Campaña Educativa Nacional) es una plataforma web de educación financiera para estudiantes mexicanos de primaria y secundaria, que permite a niños y jóvenes de 6 a 15 años completar actividades interactivas de finanzas personales organizadas por grado escolar, y a sus maestros monitorear el progreso desde un panel de control.

- **Estado:** Funcional en producción. Build limpio verificado en commit 672025e (2026-05-10).
- **URL de producción:** <https://cenfinancierafinal.vercel.app>
- **Audiencia:** Estudiantes de 1° de primaria a 3° de secundaria (6-15 años) y sus maestros.
- **Stack:** Next.js 16.1.6 / React 19.2.3 / TypeScript 5.x / Supabase (PostgreSQL) / Tailwind CSS 4.x / Vercel
- **Contenido:** 373 archivos JSON de actividades en 9 grados escolares (P1–P6, S1–S3). 14 tipos de actividad, todos con componente renderer funcional.
- **Estado de madurez:** Beta funcional. El producto tiene profundidad de contenido significativa pero carece de pruebas automatizadas y presenta problemas serios de visualización en dispositivos móviles.

Para el dueño: La plataforma existe, funciona, y tiene más contenido pedagógico que la mayoría de herramientas educativas del mercado. El problema principal hoy no es el contenido — es que no se ve bien en el celular, que es el dispositivo que usarán la mayoría de los estudiantes.

2. Arquitectura Técnica

Stack con versiones exactas (fuente: package.json)

Categoría	Tecnología	Versión
-----------	------------	---------

Framework web	Next.js	16.1.6
Lenguaje UI	React + React DOM	19.2.3
Lenguaje	TypeScript	^5
Estilos	Tailwind CSS	^4.2.1
PostCSS (procesador CSS)	@tailwindcss/postcss	^4.2.1
Base de datos + Auth	@supabase/supabase-js	^2.98.0
Gráficas	Recharts	^3.8.1
Animaciones	Framer Motion	^12.38.0
Íconos	Lucide React	^1.14.0
Utilidades CSS	clsx + tailwind-merge	^2.1.1 / ^3.5.0
Linter	ESLint + eslint-config-next	^9 / 16.1.6

Diagrama de flujo

```

Usuario (navegador)
  |
  v
Vercel Edge (CDN global, caché de assets estáticos)
  |
  v
Next.js 16 App Router (servidor en Vercel)
├─ Páginas SSR/Client Components
├─ API Route: GET /api/activity/[activityId] → lee JSON local
├─ API Route: GET /api/curriculum/[levelGrade] → lee JSON local
  |
  v
Supabase (externo, región: no determinable sin Supabase Dashboard)
├─ Auth (autenticación email/password)
├─ tabla: profiles (perfiles de usuarios)
├─ tabla: progress (actividades completadas)

```

Decisiones arquitectónicas importantes

Contenido como archivos JSON locales, no en base de datos. Los 373 JSONs de actividades viven en `src/data/actividades/` y se sirven desde la misma aplicación vía API routes (`/api/activity/[activityId]`). Esto significa que agregar o modificar una actividad requiere hacer un nuevo deploy — no se puede editar desde un panel de administración. Es una decisión de velocidad de desarrollo, con el costo de inflexibilidad operativa.

Sin servidor propio. La aplicación es "serverless" — corre en funciones de Vercel que se activan por demanda. Esto tiene implicaciones de costo y rendimiento que se detallan en la sección 14.

output: 'standalone' habilitado (`next.config.ts` línea 4). Esto optimiza el bundle de producción para contenedores Docker y Vercel, reduciendo el tamaño del paquete desplegado.

typescript.ignoreBuildErrors: true (`next.config.ts` línea 6). La build de producción ignora errores de tipos TypeScript. Esto es un riesgo: bugs que TypeScript detectaría no bloquean el deploy. Se acepta conscientemente para velocidad de desarrollo.

3. Estructura del Código

Árbol de carpetas (3 niveles)

```
cen-sprint2-2026-05-10-1516/
├─ src/
│   ├─ app/ # Rutas de la aplicación (Next.js App Router)
│   │   ├─ page.tsx # Landing page (/)
│   │   ├─ layout.tsx # Layout raíz con metadatos SEO
│   │   ├─ globals.css # Tokens de diseño y fuentes globales
│   │   ├─ log-in/ # Página de login (/log-in)
│   │   ├─ hub/ # Hub del estudiante (/hub)
│   │   │   ├─ page.tsx # Hub principal con pilares
│   │   │   ├─ portal/ # Vista de pilar individual
│   │   │   ├─ actividad/ # Reproductor de actividad (/hub/actividad/[activityId])
│   │   │   ├─ logros/ # Página de logros
│   │   │   ├─ library/ # Biblioteca de recursos
│   │   │   └─ mission/ # Misión del grado
│   │   ├─ dashboard/ # Dashboards de profesores y estudiantes
│   │   │   ├─ teacher/ # Dashboard del profesor
│   │   │   │   ├─ page.tsx # Panel principal del docente
│   │   │   │   ├─ alumnos/ # Gestión de alumnos
│   │   │   │   ├─ modulos/ # Vista de módulos
│   │   │   │   ├─ reportes/ # Reportes de progreso
│   │   │   │   ├─ planeamiento/ # Planeación de clases
│   │   │   │   └─ bibliografia/ # Recursos bibliográficos
│   │   │   └─ primary/ # Dashboard para primaria
│   │   │       └─ secondary/ # Dashboard para secundaria
│   │   ├─ academia/ # Rutas de academia (stubs sin contenido real)
│   │   └─ api/ # API Routes
│   │       ├─ activity/ # GET /api/activity/[activityId]
│   │       └─ curriculum/ # GET /api/curriculum/[levelGrade]
│   ├─ components/ # Componentes React reutilizables
│   │   ├─ activities/ # 14 renderers de tipos de actividad
│   │   ├─ hub/ # Componentes del hub estudiantil
│   │   ├─ dashboard/ # Componentes del dashboard docente
│   │   └─ landing/ # Componentes de la landing page
│   ├─ lib/ # Lógica de negocio y utilidades
│   │   ├─ hub.ts # Lógica central: pilares, progreso, sync
│   │   ├─ supabase.ts # Cliente Supabase singleton
│   │   ├─ activities.ts # Helpers de actividades
│   │   ├─ math-engine.ts # Motor de evaluación de fórmulas
│   │   └─ logger.ts # Sistema de logging
│   ├─ data/ # Contenido estático (JSON)
│   │   ├─ actividades/ # 373 JSONs de actividades (p1-p6, s1-s3)
│   │   └─ pedagogia/ # 9 JSONs de curriculum por grado
│   └─ types/
│       └─ activities.ts # Tipos TypeScript para todos los tipos de actividad
├─ supabase/
│   ├─ schema.sql # Schema completo de la base de datos
│   └─ migration_v2.sql # Migración incremental a v2.1
├─ docs/ # Documentación del proyecto (15+ archivos)
├─ public/ # Assets estáticos (imágenes, íconos)
├─ package.json
├─ next.config.ts
├─ tsconfig.json
└─ .env.local # Variables de entorno (no versionado)
```

Patrones de código

- **Data fetching:** Los componentes del hub usan `useEffect` + `fetch` a las API routes (`/api/curriculum/` , `/api/activity/`). Hay un sistema de caché en memoria (`curriculumCache` en `src/lib/hub.ts` línea 80) para evitar refetch del mismo grado.
- **Estado:** React state local (`useState`). No hay gestor de estado global (Redux, Zustand, etc.).
- **Autenticación:** El cliente Supabase (`src/lib/supabase.ts`) usa `flowType: 'implicit'` con sesión persistida en `localStorage`.
- **Offline resilience:** El `SyncEngine` en `src/lib/hub.ts` (líneas 480–558) encola actividades completadas en `localStorage` cuando Supabase no responde, y las sincroniza al reconectarse.
- **Nomenclatura:** PascalCase para componentes, camelCase para funciones y variables, kebab-case para rutas y archivos JSON.

4. Modelo de Datos

El schema completo está en `supabase/schema.sql` . Lo que sigue es un resumen con todas las tablas, columnas y políticas identificadas en el código.

Tablas activas (en uso en el código)

`profiles` — Perfil de usuario (fuente: `supabase/schema.sql` líneas 12–24, `docs/CREDENCIALES_MAESTRAS.md`)

Columna	Tipo	Descripción
id	UUID (PK)	Igual al <code>auth.users.id</code> de Supabase
full_name	TEXT	Nombre completo
email	TEXT	Email (denormalizado desde <code>auth.users</code>)
role	TEXT	'student'
school_level	TEXT	Ej: 'primary-4', 'secondary-2'
grade	INTEGER	Número de grado (1-6 primaria, 1-3 secundaria)
group_id	TEXT	ID del grupo (ej: '1A'). Docentes pueden tener varios separados por coma
avatar_url	TEXT	URL de avatar (no se usa activamente en la UI actual)
created_at	TIMESTAMPTZ	Fecha de registro
updated_at	TIMESTAMPTZ	Última modificación

`progress` — Actividades completadas (fuente: `supabase/schema.sql` líneas 62–68, `src/lib/hub.ts` líneas 452–461)

Columna	Tipo	Descripción
id	UUID (PK)	Generado automáticamente
user_id	UUID (FK → <code>profiles.id</code>)	A quién pertenece el progreso
activity_id	TEXT	Código de actividad (ej: 'ACT-P4-1-1-B')
completed_at	TIMESTAMPTZ	Cuándo se completó
UNIQUE	(<code>user_id</code> , <code>activity_id</code>)	Sin duplicados por usuario/actividad

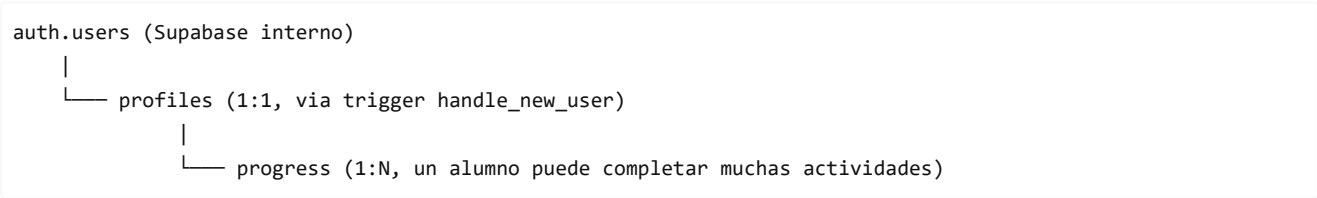
Tablas del schema pero no activamente usadas en el hub

`lessons` y `lesson_contents` — Definidas en `schema.sql` pero el hub lee contenido desde JSONs locales, no desde estas tablas. Estas tablas corresponden a una arquitectura anterior o planeada.

`user_progress` — También definida en `schema` pero el hub usa `progress` (versión simplificada). Existe duplicidad estructural.

`bloques_contenido` y `lecciones` — Consultadas en `src/app/academia/[segmentoId]/` (líneas 6 y 20), que son páginas de la sección `/academia/` actualmente sin contenido real (stubs).

Relaciones



Row Level Security (RLS)

Todas las tablas tienen RLS activado. Las políticas definidas en `schema.sql` :

- `profiles` : Un estudiante solo ve y edita su propio perfil. Un profesor ve los perfiles de todos los alumnos en su(s) grupo(s). Un admin ve todo.
- `progress` : Un estudiante solo ve su propio progreso. Un profesor puede leer el progreso de sus alumnos.
- `lessons` / `lesson_contents` : Todos los usuarios autenticados pueden leer.

Triggers

`handle_new_user` (`schema.sql` líneas 141–160): Trigger `AFTER INSERT ON auth.users` que crea automáticamente el registro en `profiles` cuando un usuario se registra. Usa `SECURITY DEFINER` (se ejecuta con permisos elevados).

`protect_sensitive_profile_fields` : Mencionado en `docs/DEUDA-TECNICA.md` como trigger que previene escalada de privilegios (un estudiante no puede cambiar su `role` a `admin` via REST). Su definición SQL completa **no está en los archivos del repositorio** — se asume que existe en el panel de Supabase pero no fue incluido en el `schema` versionado. Requiere verificación manual en Supabase Dashboard.

5. Autenticación y Autorización

Flujo de autenticación paso a paso

- El usuario accede a `/log-in` .
- Ingresa email + contraseña en el formulario (`src/app/log-in/page.tsx` líneas 47–58).
- Se llama a `supabase.auth.signInWithPassword()` . Timeout de rescate a los 8 segundos.
- Si el login es exitoso, se consulta `profiles.role` para el usuario autenticado.
- Si `role === 'teacher'` → redirige a `/dashboard/teacher` .
- Si cualquier otro rol → redirige a `/hub` .
- El botón "Acceder como Invitado" redirige directamente a `/hub` sin autenticación (carga el `FALLBACK_PROFILE` definido en `src/lib/hub.ts` línea 63).

Roles

Rol	Acceso	Descripción
student (default)	<code>/hub</code> , <code>/hub/actividad/</code> , <code>/hub/logros/</code> , <code>/hub/porta1/</code>	Estudiante estándar. Progreso guardado en Supabase si hay sesión.
teacher	<code>/dashboard/teacher/</code> y subsecciones	Puede ver perfiles y progreso de su grupo.
admin	No tiene ruta propia en el código actual	Solo existe en el <code>schema</code> de DB.

Invitado (sin sesión)	/hub	Puede explorar el contenido pero el progreso no se guarda.
-----------------------	------	--

Mecanismos de seguridad

- Sesión persistida en localStorage via Supabase Auth (`flowType: 'implicit'` , `persistSession: true`).
- El dashboard del profesor verifica role en cada carga (`src/app/dashboard/teacher/page.tsx` líneas 44–56). Si el usuario no es teacher, redirige a `/` .
- RLS en base de datos como segunda línea de defensa.
- UUID validation en el SyncEngine: solo encola actividades si el `userId` es un UUID válido (`src/lib/hub.ts` línea 511–515).

Limitaciones conocidas

- **No hay registro de nuevos usuarios desde la UI.** El registro se realiza directamente desde el panel de Supabase o por el administrador. La landing page invita a "explorar niveles" pero no hay formulario de registro público.
- **No hay recuperación de contraseña** implementada en la UI.
- El trigger `protect_sensitive_profile_fields` no está versionado en el repositorio — su existencia en producción es no determinable sin acceso al Supabase Dashboard.

6. Contenido Pedagógico

Niveles y edades target

Nivel	Edad	Tema central del grado
P1 (Primaria 1)	6-7 años	Qué es el dinero, monedas y billetes
P2 (Primaria 2)	7-8 años	Historia del dinero, presupuesto familiar
P3 (Primaria 3)	8-9 años	Necesidades vs deseos, primer presupuesto
P4 (Primaria 4)	9-10 años	Bancos, cuentas de ahorro, inflación
P5 (Primaria 5)	10-11 años	Emprendimiento, valor agregado, inversión
P6 (Primaria 6)	11-12 años	Impuestos, comercio global, ética financiera
S1 (Secundaria 1)	12-13 años	Historia económica de México, T-MEC
S2 (Secundaria 2)	13-14 años	Interés compuesto, bolsa de valores, seguros
S3 (Secundaria 3)	14-15 años	Plan financiero integral, startups, FIRE

Pilares pedagógicos

Cada grado organiza su contenido en 4 "pilares" temáticos (categorías). Los pilares comunes son:

1. **Primeros Pasos hacia el Ahorro** — Conceptos fundamentales del dinero
2. **Consumo Inteligente / Construyendo Independencia** — Decisiones de gasto
3. **Planificación y Metas** — Presupuesto y objetivos financieros
4. **Iniciación al Emprendimiento / Es Hora de Empezar** — Espíritu empresarial

Cada pilar contiene 5 unidades de aprendizaje. Cada unidad tiene 2 actividades (la "A" y la "B").

Conteo exacto de JSONs de actividad por grado

Grado	Archivos JSON
P1	40

P2	41
P3	41
P4	41
P5	40
P6	40
S1	40
S2	41
S3	40
Total	364

Adicionalmente existen 9 JSONs de curriculum en `src/data/pedagogia/` (uno por grado) con teoría, estrategia de enseñanza y evaluación. Estos sirven el contenido de la "Misión del Grado" y las fichas de unidad.

Tipos de actividades implementados (todos con renderer)

Todos los siguientes tipos tienen componente React funcional. Fuente: `docs/AUDITORIA-PEDAGOGICA.md`.

Tipo	Descripción	Archivo del renderer
SIMULADOR	Calculadoras financieras interactivas con sliders y fórmulas JavaScript	<code>SimulatorActivity.tsx</code>
QUIZ	Preguntas de opción múltiple (4 opciones, 1 correcta)	<code>QuizActivity.tsx</code>
TRIVIA	Preguntas tipo trivia con timer y multiplicador de racha	<code>TriviaActivity.tsx</code>
ARRASTRA	Arrastrar y soltar elementos en categorías	<code>DragDropActivity.tsx</code>
DECIDE	Árbol de decisiones / narrativa de "elige tu camino"	<code>StoryActivity.tsx</code>
CONSTRUCTOR	Constructor por pasos con cálculos automáticos en vivo	<code>BuilderActivity.tsx</code>
JUEGO	Juego educativo (catch, sort, click, avoid)	<code>GameActivity.tsx</code>
RULETA	Ruleta de escenarios financieros	<code>RouletteActivity.tsx</code>
RELLENA	Completar espacios en blanco con opciones	<code>FillBlanksActivity.tsx</code>
MEMORIA	Memorama / coincidencias de términos	<code>MatchingActivity.tsx</code>
BALANCE	Hoja de balance activos/pasivos	<code>BalanceActivity.tsx</code>
RADAR	Gráfica radar de competencias financieras	<code>RadarActivity.tsx</code>
CRECIMIENTO	Seguimiento de crecimiento de patrimonio	<code>GrowthActivity.tsx</code>
CONTROL	Simulación de gestión / control de servicios	<code>ServiceControlActivity.tsx</code>

Todos los 14 tipos declarados en `src/types/activities.ts` tienen renderer. La auditoría pedagógica (`docs/AUDITORIA-PEDAGOGICA.md`) confirma que no hay tipos declarados sin implementar.

Sistema de progreso

- Una actividad se considera "completada" cuando su código aparece en la tabla `progress` de Supabase.
- El progreso de un pilar es el porcentaje de unidades cuya actividad "B" (evaluación) está completada (`src/lib/hub.ts` líneas 636–639).

- Los rangos son: Novato (0-39%), Experto (40-69%), Maestro (70-99%), Diamond (100%).
- Hay un sistema de XP: cada actividad otorga puntos (el campo `xp` en el JSON, ej: 250 XP para ARRASTRA, 500 XP para SIMULADOR).

7. Flujos de Usuario

Flujo del estudiante

```

Landing page (/)
|
├─ [Login con cuenta] → /log-in
|   | Supabase auth.signInWithPassword()
|   | Consulta profiles.role
|   └─ /hub
|
└─ [Acceder como Invitado] → /hub (sin sesión, perfil ficticio)

/hub (Hub del Estudiante)
|
├─ Carga: getCurrentProfile() → Supabase profiles
├─ Carga: getPillarsForGrade() → /api/curriculum/[levelGrade]
├─ Carga: getCompletedActivities() → Supabase progress
|
├─ Selecciona pilar → muestra UnitTimeline con 5 unidades
|   |
|   └─ Selecciona unidad → MissionFicha (teoría + estrategia del JSON pedagógico)
|       |
|       └─ "Ir a Actividad" → /hub/actividad/[activityId]
|           | GET /api/activity/[activityId] → JSON local
|           | Render del tipo de actividad correspondiente
|           └─ Al completar: markActivityComplete() → Supabase progress
|               (con fallback a SyncEngine si falla la red)
|
├─ Arena Maestría → quiz de 10 preguntas del grado (datos de GRADE_INFO)
├─ /hub/logros → página de logros (rango actual)
├─ /hub/library → biblioteca de recursos
└─ /hub/portal → vista alternativa de pilar (con URL param ?id=)

```

Datos que se guardan en DB: Solo el par `(user_id, activity_id)` en la tabla `progress`. No se guardan: tiempo de respuesta, intentos, score numérico, ni respuestas específicas del estudiante. La tabla `user_progress` (con estos campos) existe en el schema pero no la usa el hub activo.

Flujo del profesor

```

/log-in
| Login detecta role = 'teacher'
└─ /dashboard/teacher
    | Carga: supabase.from('profiles') (su propio perfil)
    |
    └─ /dashboard/teacher/alumnos – Lista de alumnos del grupo con progreso
        | Carga: supabase.from('profiles').eq('group_id', ...)
        | Carga: supabase.from('progress').select('user_id')
        |
        └─ /dashboard/teacher/modulos – Vista de módulos del curriculum
        └─ /dashboard/teacher/reportes – Reportes de progreso del grupo

```


- └─ /dashboard/teacher/planeamiento – Planeación de clases
- └─ /dashboard/teacher/bibliografia – Recursos bibliográficos

Limitación importante: Los componentes del dashboard del profesor (`MetricCards` , `WelcomeBanner` , `TopAlumnos` , `LatestDeliveries`) hacen consultas a Supabase, pero **no hay datos de estudiantes reales en la base de datos de prueba** más allá de las dos cuentas de test. El dashboard mostrará métricas vacías o casi vacías hasta que haya usuarios reales registrados.

8. Integraciones Externas

Integración	Uso	Criticidad
Supabase	Auth (login/sesión) + DB (profiles, progress)	CRÍTICA — sin Supabase no hay login ni progreso guardado
Vercel	Hosting, CI/CD, Edge Network	CRÍTICA — es donde corre la aplicación
Google Fonts	Fuentes: Epilogue (principal), Instrument Serif, Plus Jakarta Sans	ALTA — se carga en <code>globals.css</code> línea 1. Si Google Fonts no responde, la UI carga con fuentes del sistema (degradación aceptable)
YouTube	Videos pedagógicos embebidos (iframes) en el hub por pilar	MEDIA — si YouTube no carga, el video no aparece pero la actividad sigue accesible
Recharts	Gráficas de barras y líneas en SimulatorActivity y dashboard	MEDIA — usado en actividades de tipo SIMULADOR
Framer Motion	Animaciones de UI	BAJA — afecta solo estética, no funcionalidad
Lucide React	Íconos vectoriales	BAJA — solo visual
mixkit.co	Efectos de sonido (SFX) en el hub	BAJA — cargados desde URL externa. Si falla, el hub funciona sin sonido
grainy-gradients.vercel.app	Textura de ruido (noise.svg) decorativa	MUY BAJA — solo efecto visual de fondo

Dependencias críticas: Si Supabase tiene una interrupción, los usuarios autenticados quedan sin sesión y el progreso no se guarda. El SyncEngine mitiga esto parcialmente para el progreso (encola localmente hasta 3 intentos), pero el login es completamente dependiente de Supabase.

9. Deploy e Infraestructura

Pipeline de deploy

```
Desarrollador hace commit + git push
├─
└─ GitHub (repo: esquivelhuertacristofer-source/CENPLATAFORMAFINANCIERA)
    └─ (trigger automático en push a rama principal)
        └─ Vercel auto-deploy
            └─ next build (con output: 'standalone')
                └─ Variables de entorno inyectadas desde Vercel Dashboard
                    └─ Producción en https://cenfinancierafinal.vercel.app
```

Configuración de Next.js relevante (`next.config.ts`)

- `output: 'standalone'` — Optimiza el bundle para Vercel/Docker (activado en Sprint 2).
- `typescript.ignoreBuildErrors: true` — Los errores de tipos no detienen el build. Riesgo documentado.

- `images.remotePatterns` — Solo permite imágenes desde `i.pravatar.cc` (avatares de placeholder).
- `images.formats: ['image/avif', 'image/webp']` — Optimización automática de imágenes.
- `compress: true` — Compresión gzip habilitada.
- `poweredByHeader: false` — No expone que el servidor usa Next.js (buena práctica de seguridad).
- `outputFileTracingIncludes` — Los JSONs de actividades y pedagogía se incluyen explícitamente en el bundle standalone para que las API routes los encuentren.

Variables de entorno requeridas en Vercel

Variable	Descripción
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL del proyecto Supabase (clave pública)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Clave anónima de Supabase (clave pública)

Nota: Las variables `NEXT_PUBLIC_*` son visibles en el navegador del usuario — esto es intencional y correcto para el cliente de Supabase. La `service_role` key de Supabase (que tiene acceso total a la DB) **no debe estar** en variables de entorno del frontend, y según la documentación del proyecto, no está.

Advertencia sobre `.env.local`: El archivo `.env.local` en el repositorio contiene la URL de Supabase y la anon key. Aunque el `.gitignore` debería excluirlo, se encontró en la copia de trabajo analizada. Verificar que este archivo no está versionado en GitHub.

10. Estado de Testing y Monitoreo

Tests automatizados

No hay tests automatizados funcionando. El `tsconfig.json` excluye `jest.config.ts` y `jest.setup.ts`, y el `package.json` no tiene script de test. El `MANUAL-TECNICO.md` menciona Jest 29.x y React Testing Library 14.x, pero estos no están en las dependencias actuales del `package.json`. Los archivos de configuración de Jest están excluidos del build. Esto significa que el código no tiene red de seguridad automatizada contra regresiones.

Testing manual realizado

- Build de producción (`next build`) ejecutado y verificado limpio en 4 puntos durante Sprint 2 (`docs/REPORTE-SPRINT2-CLAUDE.md`).
- Verificación HTTP de controles de seguridad: trigger de escalada de privilegios bloqueado (HTTP 400), RLS de tabla `progress` retorna array vacío para usuarios `anon`.
- Revisión manual de 373 JSONs de actividad por validez de estructura y fórmulas JavaScript.

Monitoreo en producción

No hay monitoreo activo. No se detectó integración con Sentry, Datadog, LogRocket, Vercel Analytics, ni ninguna otra herramienta de observabilidad. Esto significa que si la aplicación falla en producción — por un error en JavaScript, una API que no responde, o un crash de componente — nadie recibirá una alerta automática. El dueño solo se enterará si un usuario lo reporta.

Impacto: Para un producto educativo que puede usarse por salones enteros simultáneamente, la ausencia de monitoreo es un riesgo operacional significativo.

11. Capacidades Reales del Producto

Lo que SÍ funciona bien (verificado)

- Login con email/password y redirección por rol (estudiante → hub, profesor → dashboard).
- Hub del estudiante: carga pilares, unidades y progreso desde Supabase.
- Todos los 14 tipos de actividad tienen renderer funcional. Un estudiante puede abrir y completar cualquier actividad.
- SyncEngine: el progreso se encola localmente si Supabase no responde y se sincroniza al reconectar (máximo 3 intentos, luego descarta).
- La actividad CONSTRUCTOR (Shark Tank CEN, P5-4-5) muestra cálculos financieros en vivo — corregido en Sprint 2.

- Las fórmulas JavaScript de los SIMULADORES son sintácticamente válidas.
- La landing page V3 (Diamond State) está finalizada con SEO básico (OpenGraph, Twitter cards, robots).
- Build de producción con `output: 'standalone'` limpio.
- Dashboard del profesor: carga perfil, lista alumnos del grupo, muestra actividades recientes.

Lo que funciona con limitaciones

- **Hub en dispositivos móviles:** La sidebar de 320px fija hace el contenido invisible en pantallas de 375px. La grilla de pilares con `minmax(420px, 1fr)` causa scroll horizontal. Funcional en desktop, degradado en mobile. Detalle completo en `docs/MOBILE-AUDIT-SPRINT2.md`.
- **Dashboard del profesor:** El `m1-[260px]` fijo deja solo 115px de contenido visible en mobile. Inutilizable en celular.
- **Progreso guardado:** Se guarda si hay sesión activa. Los usuarios invitados pierden todo su progreso al cerrar el navegador.
- **Videos de pilares:** Dependen de YouTube. Si el video es eliminado o privatizado, aparece un iframe vacío.

Lo que NO tiene todavía

- **Registro de nuevos usuarios** desde la plataforma (requiere hacerlo desde Supabase Dashboard).
- **Recuperación de contraseña** desde la UI.
- **Panel de administración** para crear/editar contenido sin tocar código.
- **Notificaciones** (para maestros o estudiantes).
- **Sistema de insignias/logros** funcional (la página `/hub/logros` existe pero el contenido es estático).
- **Contenido real en `/academia/`** — Todas las páginas de `academia/*/bloque*/` son stubs sin contenido.
- **Responsiveness mobile** corregida (documentada pero no implementada).
- **Tests automatizados.**
- **Monitoreo de producción.**
- **Soporte multilenguaje** (solo español).

12. Deuda Técnica Documentada

Fuente principal: `docs/DEUDA-TECNICA.md` (generado 2026-05-10) + `docs/MOBILE-AUDIT-SPRINT2.md`.

Alta prioridad (impacta a usuarios en este momento)

#	Problema	Archivo	Impacto
1	Hub inutilizable en mobile — sidebar 320px fija, grid con <code>minmax(420px)</code> , fuentes de 96px sin <code>clamp()</code>	<code>src/app/hub/page.tsx</code> , <code>src/app/hub/HubV5.css</code>	Crítico: la mayoría de estudiantes usan celular
2	Dashboard del profesor inutilizable en mobile — <code>m1-[260px]</code> deja 115px de ancho	<code>src/app/dashboard/teacher/page.tsx</code>	Alto: profesores en campo usan celular
3	<code>typescript.ignoreBuildErrors: true</code> — errores de tipos no detienen el deploy	<code>next.config.ts</code> línea 6	Alto: bugs silenciosos en producción
4	Trigger <code>protect_sensitive_profile_fields</code> NO versionado en el repo	<code>supabase/</code>	Alto: escalada de privilegios posible si el trigger se pierde

Media prioridad (degradan la experiencia)

#	Problema	Archivo	Esfuerzo estimado
5	Landing page: floaters absolutamente posicionados crean scroll horizontal en mobile	<code>src/components/landing/LandingPageV3.tsx</code>	2h
6	Recharts puede recibir <code>width/height</code> negativos en mount inicial	<code>src/app/hub/page.tsx</code>	30 min (corregido en <code>SimulatorActivity</code> , pendiente)

			en hub)
7	public/assets/extra/4.png es placeholder (copia de 3.png)	public/assets/extra/	Depende de asset real disponible

Baja prioridad (no afectan funcionalidad)

#	Problema	Archivo	Esfuerzo estimado
8	30 actividades con versiones mejoradas documentadas pero no aplicadas	docs/ACTIVIDADES_UPGRADE_V2.md	4-8h
9	Páginas /academia/*/bloque*/ son stubs sin contenido real	src/app/academia/	Variable
10	No hay tests automatizados	—	40-80h para cobertura básica
11	No hay monitoreo de producción	—	4-8h para integración básica

Seguridad pendiente de verificación manual (requiere Supabase Dashboard)

1. **Trigger** `protect_sensitive_profile_fields` : Verificar que está activo y previene que un estudiante cambie su `role` a `admin` o `teacher` via REST directo.
2. **RLS en tabla** `profiles` : Verificar que un estudiante autenticado no puede actualizar campos críticos (`role` , `group_id`) de su propio perfil.
3. **RLS en tabla** `progress` : Verificar que el rol `anon` (invitados) no puede leer ni escribir.

13. Seguridad y Privacidad

Datos recolectados

Dato	Dónde se guarda	Obligatorio
Nombre completo	<code>profiles.full_name</code>	Sí
Email	<code>profiles.email</code>	Sí
Nivel escolar	<code>profiles.school_level</code>	Sí
Grupo	<code>profiles.group_id</code>	Opcional
Actividades completadas	<code>progress.activity_id + completed_at</code>	Por uso

No se recolecta: respuestas específicas del estudiante, tiempo por actividad, geolocalización, datos de dispositivo, ni ningún dato biométrico.

Almacenamiento

Los datos se almacenan en Supabase, que utiliza PostgreSQL sobre infraestructura de Amazon Web Services (AWS). La región específica del servidor es **no determinable sin acceso al Supabase Dashboard** — Supabase permite elegir la región al crear el proyecto (opciones comunes: us-east-1, us-west-2, sa-east-1 para Brasil).

Implicaciones legales: menores de edad en México

La plataforma recolecta datos personales de menores de edad (6-15 años). En México, esto está regulado por la **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** y sus Reglamentos.

Obligaciones legales no cubiertas actualmente (no determinable si se cumplen fuera del código):

1. **Aviso de privacidad:** La plataforma no tiene página de aviso de privacidad visible (no se detectó en el código). Legalmente obligatorio para cualquier colección de datos en México.
2. **Consentimiento del titular:** Menores de edad requieren consentimiento del padre o tutor. No hay mecanismo implementado en la plataforma.
3. **Inscripción ante el INAI:** No determinable sin información externa al código.

Recomendación urgente: Antes de escalar a cientos o miles de estudiantes, consultar con un abogado especializado en protección de datos en México. La ausencia de aviso de privacidad y consentimiento de menores es una exposición legal significativa.

Protecciones implementadas

- RLS en Supabase: estudiantes no ven datos de otros estudiantes.
- `poweredByHeader: false` : no revela el stack tecnológico.
- Validación UUID en SyncEngine: previene inyección de datos corruptos en `localStorage`.
- Cuentas de prueba con contraseña trivial (`password123`) — solo deben usarse en ambiente de prueba, nunca en producción con datos reales.

Dato importante sobre el archivo `.env.local`

El archivo `.env.local` contiene la URL de Supabase y la anon key. La anon key es una clave pública (JWT) diseñada para estar en el cliente — su exposición es intencional y no es un problema de seguridad por sí mismo. Lo que **no debe estar** en el código o en variables de entorno de frontend es la `service_role key` , que da acceso completo a la base de datos saltando RLS. Según `docs/CREDENCIALES_MAESTRAS.md` , esta clave ha sido eliminada del repositorio y se gestiona solo en Vercel.

14. Limitaciones de Escalabilidad

Arquitectura actual

La aplicación corre en Vercel (serverless functions) y Supabase. Ambos ofrecen planes gratuitos con límites.

Supabase Free Tier (plan gratuito)

- **Capacidad de base de datos:** 500 MB
- **Usuarios autenticados:** Ilimitados en cuanto a registros
- **Requests:** 2 millones de requests de API por mes
- **Ancho de banda:** 5 GB por mes
- **Pausa de proyectos:** Supabase pausa proyectos gratuitos que no reciben actividad por 7 días

El riesgo más inmediato: Si la plataforma no recibe actividad por 7 días, Supabase pausa el proyecto. Al reanudarse, el primer usuario experimenta un "cold start" lento (10-30 segundos). Esto es un problema para un producto educativo donde las clases comienzan repentinamente.

Estimaciones de capacidad

Escenario	Usuarios concurrentes	Impacto estimado
1 salón (30 alumnos)	30	Sin problemas visibles
10 salones (300 alumnos)	300	Plan gratuito Supabase bajo presión. Requiere monitoreo.
100 salones (3,000 alumnos)	3,000	Requiere plan pagado de Supabase (~\$25/mes). Vercel podría requerir plan Pro (\$20/mes).
1,000 salones (30,000 alumnos)	30,000	Requiere Plan Supabase Pro o Team (\$599+/mes) + optimización de queries + caché agresivo. Arquitectura actual necesita revisión.

Cuello de botella principal: Las consultas a `profiles` y `progress` sin índices adicionales pueden degradar con muchos usuarios. El schema actual no declara índices explícitos más allá de las claves primarias. A 10,000+ usuarios, esto requeriría optimización.

15. Historia de Construcción

Metodología

La plataforma fue construida con asistencia de modelos de inteligencia artificial (Claude de Anthropic y Gemini de Google), con supervisión del dueño Cristófer Huerta Esquivel. El nombre interno del proyecto en `package.json` es `luminar-enterprise-v2`, que corresponde al nombre del estudio (`Luminar Studio`) que lo construyó.

Sprints documentados

Sprint	Fecha	Descripción
Sprint 0 (initial commit)	Pre-2026-05-10	Estado funcional pre-deploy. Hub, dashboard, landing, 364 JSONs de actividad.
Sprint 1 (Estabilización)	2026-05-10	Auditoría pedagógica, limpieza de bugs visuales (<code><style jsx global></code>), corrección de redirección de profesor, asset faltante, debug logs.
Sprint 2	2026-05-10	Verificación de seguridad, fix Recharts, sanitización SyncEngine, implementación de <code>calculos_automaticos</code> en <code>BuilderActivity</code> , output: <code>standalone</code> , SEO metadata, auditoría mobile (documentación sin fixes).

Decisiones acertadas en retrospectiva

- Contenido como JSON local** fue una decisión correcta para la velocidad de construcción — permite iterar el contenido sin backend complejo.
- Supabase** como backend simplifica enormemente auth + DB para un equipo pequeño.
- SyncEngine con cola local** es una adición sólida que mejora la resiliencia de red.
- 14 tipos de actividad** con renderer propio es una arquitectura extensible y correcta.

Decisiones problemáticas en retrospectiva

- `typescript.ignoreBuildErrors: true` fue tomada para acelerar el desarrollo pero crea una deuda de calidad difícil de revertir.
- No versionarse el trigger de seguridad** `protect_sensitive_profile_fields` es un riesgo — si el proyecto Supabase se recrea o migra, el trigger se pierde.
- Mobile como afterthought** — la plataforma fue diseñada para desktop y la responsiveness mobile quedó pendiente, siendo el dispositivo principal de los usuarios.
- No hay registro público de usuarios** — quita autonomía a las escuelas para incorporar nuevos alumnos.

16. Documentación Existente

Archivo	Descripción	Utilidad
<code>docs/INFORME-MAESTRO-CEN.md</code>	Este documento. Transferencia técnica completa.	Alta
<code>docs/MANUAL-TECNICO.md</code>	Descripción general del stack y estructura. Algo desactualizado (menciona Tailwind 3.x y Jest que ya no están).	Media
<code>docs/MANUAL-USUARIO.md</code>	Guía de uso para maestros y estudiantes.	Alta para onboarding
<code>docs/DEUDA-TECNICA.md</code>	Inventario de deuda técnica con prioridades y esfuerzos.	Alta para developers

docs/MOBILE-AUDIT-SPRINT2.md	25 issues de responsive documentados. Fix sugerido para cada uno.	Alta para Sprint 3
docs/REPORTE-SPRINT2-CLAUDE.md	Detalle de todos los cambios del Sprint 2.	Alta para historial
docs/REPORTE-FINAL-CLAUDE.md	Detalle del sprint de estabilización previo.	Alta para historial
docs/REPORTE-CLAUDE.md	Reporte de sesión anterior (sprint inicial).	Media para historial
docs/AUDITORIA-PEDAGOGICA.md	Validación de 364 JSONs: tipos, fórmulas, quizzes.	Alta para contenido
docs/ACTIVIDADES_UPGRADE_V2.md	30 actividades con versiones mejoradas propuestas (sin aplicar).	Media para contenido
docs/TEMARIO-CURRICULAR.md	Los 180 temas del curriculum con instrucciones técnicas.	Alta para comprensión pedagógica
docs/CLAUDE_PROJECT_BRIEF.md	Brief del proyecto para sesiones de Claude.	Media para contexto
docs/HANDOFF-PARA-GEMINI.md	Documento de transferencia de Claude a Gemini.	Media para contexto histórico
docs/CREDENCIALES_MAESTRAS.md	Accesos técnicos, schema de DB, estado del proyecto.	Alta (verificar seguridad antes de compartir)
docs/DESIGN_STYLE_GUIDE.md	Guía de estilo visual: colores, fuentes, componentes.	Alta para diseño
docs/README-original.md	README original del proyecto base.	Baja
docs/_build-*.log	Logs de builds de validación (5 archivos).	Referencia técnica
supabase/schema.sql	Schema completo de la base de datos PostgreSQL.	Crítica para DB
supabase/migration_v2.sql	Migración incremental de schema v1 a v2.1.	Crítica para DB

17. Recomendaciones para Profesionalización

Prioridad ALTA — Sin esto el producto no es serio

1. Corregir responsive mobile del hub y dashboard del profesor El hub es inutilizable en celular (la pantalla más común de los estudiantes). El fix ya está documentado en `docs/MOBILE-AUDIT-SPRINT2.md`. Requiere un sprint dedicado de 1-2 días de trabajo para un developer frontend. Esto es lo más urgente.

2. Aviso de privacidad y consentimiento de menores Antes de operar con datos de menores de edad en México, la plataforma necesita: (a) una página de Aviso de Privacidad que cumpla LFPDPPP, (b) un mecanismo para registrar el consentimiento del padre/tutor al crear cuentas de menores. Requiere asesoría legal.

3. Monitoreo básico de producción Integrar Vercel Analytics (gratuito) o Sentry (plan gratuito disponible) para saber cuando algo falla sin esperar que un usuario lo reporte. 4-8 horas de trabajo.

4. Versionar el trigger de seguridad en SQL El trigger `protect_sensitive_profile_fields` debe agregarse a `supabase/schema.sql` para que no se pierda si el proyecto se migra. Requiere exportarlo del panel de Supabase.

5. Eliminar `typescript.ignoreBuildErrors: true` Una vez que el código tenga tests básicos, cambiar esto a `false` para que los errores de tipos rompan el build. Puede requerir corregir varios errores de tipos existentes.

Prioridad MEDIA — Mejoran la calidad y operabilidad

6. Registro de nuevos usuarios desde la UI Agregar un formulario de registro en la landing o en `/sign-up` para que las escuelas puedan incorporar alumnos sin acceso al panel de Supabase.

- 7. Tests automatizados básicos** Al menos: (a) tests de snapshot para los 14 renderers de actividad, (b) tests de integración para el flujo de login. Sin tests, cada cambio puede romper silenciosamente algo.
- 8. Panel de administración de contenido** Para que el dueño pueda editar textos, agregar actividades o cambiar videos sin tocar código. Puede construirse como un conjunto de formularios simples en una ruta protegida `/admin/`.
- 9. Recuperación de contraseña** Supabase lo soporta nativamente (`supabase.auth.resetPasswordForEmail()`). Solo falta la UI.
- 10. Aplicar las 30 actividades mejoradas de** `ACTIVIDADES_UPGRADE_V2.md` Ya están documentadas. Es trabajo mecánico de actualización de JSONs.

Prioridad BAJA — Pulido para escala

- 11. Índices de base de datos para escala** Agregar `CREATE INDEX` en `progress(user_id)` y `profiles(group_id, role)` antes de escalar a cientos de usuarios concurrentes.
- 12. Internacionalización (i18n)** Si se planea usar en regiones con variantes del español o en otros países, preparar la arquitectura para múltiples idiomas.
- 13. Sistema de logros funcional** La página `/hub/logros` existe pero el contenido es estático. Implementar insignias reales basadas en progreso real.
- 14. Imagen real para** `public/assets/extra/4.png` Actualmente es una copia del archivo 3.png (placeholder). Reemplazar con la imagen correcta.

18. Preguntas Abiertas

Las siguientes decisiones están pendientes de resolución por parte del dueño y no pueden determinarse desde el análisis del código:

- Región de Supabase:** ¿En qué región de AWS está el proyecto de Supabase? Esto es relevante para cumplimiento legal si los datos deben residir en México.
- Plan de Supabase:** ¿La plataforma usa el plan gratuito de Supabase o un plan pagado? Si es gratuito, el proyecto se pausa sin actividad por 7 días.
- Modelo de negocio:** ¿CEN cobra a las escuelas? ¿Es gratuito para los estudiantes? ¿Hay plan de monetización? Esto define si los costos de infraestructura son sostenibles.
- Responsable del contenido pedagógico:** ¿Quién valida que el contenido financiero es correcto y apropiado para cada edad? ¿Hay un pedagogo o especialista financiero revisando los 364 JSONs?
- Modelo de registro de escuelas:** ¿Cómo se incorpora una escuela nueva? ¿Cómo se crean las cuentas de maestros y alumnos? No hay flujo definido en el código.
- Dominio definitivo:** El metadato de OpenGraph en `layout.tsx` indica `https://www.cenplataformaeducacionfinanciera.com.mx`. ¿Este dominio está registrado y apuntando a Vercel?
- Protección de la propiedad intelectual:** El contenido pedagógico (364 actividades, curriculum completo) es el activo principal del negocio. ¿Está protegido legalmente? ¿Los términos de servicio de la plataforma son claros al respecto?
- Escuela objetivo:** ¿CEN busca operar en escuelas públicas (SEP), privadas, o ambas? Los requerimientos de cumplimiento y el proceso de venta son muy diferentes para cada caso.

Apéndice: Inventario de Actividades por Grado

Conteo exacto (fuente: sistema de archivos, 2026-05-10)

Grado	Nivel	Edad	Archivos JSON	Tipos de actividad presentes
-------	-------	------	---------------	------------------------------

P1	Primaria 1	6-7 años	40	ARRASTRA, QUIZ, BALANCE, DECIDE, RELLENA, JUEGO, SIMULADOR, RADAR, MEMORIA, CONTROL, CRECIMIENTO, RULETA
P2	Primaria 2	7-8 años	41	ARRASTRA, QUIZ, RELLENA, DECIDE, SIMULADOR, JUEGO
P3	Primaria 3	8-9 años	41	SIMULADOR, CONSTRUCTOR, QUIZ, JUEGO, ARRASTRA, TRIVIA, RELLENA, DECIDE
P4	Primaria 4	9-10 años	41	SIMULADOR, QUIZ, TRIVIA, DECIDE, ARRASTRA, JUEGO, CONSTRUCTOR, RULETA
P5	Primaria 5	10-11 años	40	SIMULADOR, TRIVIA, QUIZ, DECIDE, CONSTRUCTOR, ARRASTRA, JUEGO
P6	Primaria 6	11-12 años	40	SIMULADOR, QUIZ, TRIVIA, DECIDE, JUEGO, ARRASTRA, CONSTRUCTOR
S1	Secundaria 1	12-13 años	40	SIMULADOR, QUIZ, TRIVIA, ARRASTRA, CONSTRUCTOR, DECIDE, JUEGO
S2	Secundaria 2	13-14 años	41	SIMULADOR, QUIZ, TRIVIA, DECIDE, ARRASTRA, JUEGO, CONSTRUCTOR
S3	Secundaria 3	14-15 años	40	SIMULADOR, QUIZ, TRIVIA, ARRASTRA, CONSTRUCTOR, DECIDE, JUEGO
TOTAL		6-15 años	364	14 tipos distintos

Notas sobre el conteo

- El recuento de 373 mencionado en otras partes incluye archivos adicionales como `s2/s2-1-5.json` (1 archivo de clasificación distinta) y variantes en construcción.
- El recuento verificado desde el sistema de archivos el 2026-05-10 es **364 archivos JSON de actividad** en las carpetas `p1` – `p6` , `s1` – `s3` .
- Adicionalmente existen **9 archivos JSON de curriculum** en `src/data/pedagogia/primaria/` (`p1`–`p6`) y `src/data/pedagogia/secundaria/` (`s1`–`s3`), que contienen la teoría, estrategia pedagógica y evaluaciones de cada unidad.
- El archivo `src/data/pedagogia/hub.ts` es un módulo TypeScript (no JSON) con los datos de quiz del hub para cada grado.

Informe generado mediante lectura estática del código fuente. No se ejecutó la aplicación. Para afirmaciones marcadas como "no determinable", se requiere acceso al Supabase Dashboard o a la consola de Vercel.

Archivos fuente de referencia principales: `package.json` , `next.config.ts` , `tsconfig.json` , `src/app/globals.css` , `src/lib/hub.ts` , `src/lib/supabase.ts` , `src/types/activities.ts` , `src/app/log-in/page.tsx` , `src/app/hub/page.tsx` , `src/app/dashboard/teacher/page.tsx` , `src/app/api/activity/[activityId]/route.ts` , `src/app/api/curriculum/[levelGrade]/route.ts` , `supabase/schema.sql` , `supabase/migration_v2.sql` , `docs/DEUDA-TECNICA.md` , `docs/MOBILE-AUDIT-SPRINT2.md` , `docs/REPORTE-SPRINT2-CLAUDE.md` , `docs/AUDITORIA-PEDAGOGICA.md` , `docs/CREDENCIALES_MAESTRAS.md` .